

Guía de Exposición — Firebase & Despliegue en Render

Documento de Sustentación Técnica para Exposición

UNAMConnect — Portal de Tutorías Académicas

Esta guía está diseñada para respaldar la explicación técnica sobre la integración de **Google Firebase** y el proceso de despliegue continuo (**CI/CD**) en la nube de **Render** durante la sustentación ante el jurado o docente.

1. Introducción y Arquitectura Cloud ☁

El sistema **UNAMConnect** adopta una arquitectura basada en **Servicios en la Nube (PaaS & Serverless)** para garantizar disponibilidad 24/7, escalabilidad y seguridad de almacenamiento de archivos sin sobrecargar el servidor principal.

Componentes Cloud Principales:

1. Google Firebase Console & SDK:

- * **Firebase Auth**: Gestión de identidades y enlace seguro para recuperación de claves.
- * **Firebase Admin SDK**: Control administrativo en el backend Node.js y autenticación con llaves públicas rotativas de Google.
- * **Firebase Storage**: Almacenamiento de archivos pesados (boletas de notas en PDF, historiales académicos e imágenes).

2. Plataforma como Servicio — Render (Render Cloud Platform):

- * **Render Web Services**: Alojamiento del backend Node.js / Express y del frontend en Angular.
- * **Automatización CI/CD**: Despliegue continuo automático conectado al repositorio oficial de GitHub (<https://github.com/JeffCortez23/UNAMConnect>).

2. Implementación de Google Firebase 🌐

2.1. Firebase Authentication

¿Por qué se eligió?: Permite delegar la seguridad del restablecimiento de contraseñas y la emisión de tokens a la infraestructura global de Google. Evita tener que implementar servidores SMTP complejos en desarrollo local para enviar correos de verificación.

Operación en Frontend (`firebase.service.ts`):

```
`typescript

async sendPasswordReset(email: string): Promise {

await sendPasswordResetEmail(auth, email);

}

`
```

Al solicitar el cambio de clave, Firebase Auth envía de forma automática e inmediata un correo seguro al estudiante con un enlace con token de vida corta.

2.2. Firebase Admin SDK (Backend Node.js)

Archivo Clave: `backend/services/firebase.service.js`

Inicialización Híbrida (Desarrollo vs Producción):

El backend implementa un mecanismo inteligente de inicialización:

* **En Entorno Local:** Carga la clave privada desde el archivo físico `backend/config/firebase-service-account.json` .

* **En Producción (Render):** Lee las credenciales directamente de las variables de entorno del servidor (`FIREBASE_PROJECT_ID` , `FIREBASE_CLIENT_EMAIL` , `FIREBASE_PRIVATE_KEY`), procesando los saltos de línea con `.replace(/\n/g, '\n')` .

```
`javascript

if (process.env.FIREBASE_PROJECT_ID && process.env.FIREBASE_CLIENT_EMAIL &&
process.env.FIREBASE_PRIVATE_KEY) {

admin.initializeApp({

credential: credential.cert({

projectId: process.env.FIREBASE_PROJECT_ID,

clientEmail: process.env.FIREBASE_CLIENT_EMAIL,

privateKey: process.env.FIREBASE_PRIVATE_KEY.replace(/\n/g, '\n')

}))

});

}
```

2.3. Firebase Storage (Almacenamiento de Boletas y PDFs)

¿Por qué se eligió?: Los servidores de aplicación (como Render o Heroku) tienen sistemas de archivos efímeros (*Ephemeral File Systems*), lo que significa que los archivos subidos al disco local se pierden cuando el servidor se reinicia. Firebase Storage almacena los documentos en buckets de la nube con redundancia geográfica.

Flujo de Subida:

1. El estudiante selecciona su Boleta de Notas en PDF en el formulario de postulación a tutor.
2. El cliente invoca a `FirestoreService.uploadFile(path, file)`.
3. El archivo se sube directamente a Firebase Storage (`uploadBytes`).
4. Firebase genera una URL pública firmada (`getDownloadURL`) que se envía al backend Express y se persiste en PostgreSQL.

3. Proceso de Despliegue en Render (Render Cloud Platform) 🚀

El despliegue en producción se realiza en la plataforma PaaS **Render**, la cual ofrece integración directa con GitHub y soporte nativo para Node.js y entornos web.

3.1. Arquitectura de Despliegue y Enlace en Producción

URL de Producción: `https://unamconnect.onrender.com`

Repositorio GitHub: `https://github.com/JeffCortez23/UNAMConnect`

3.2. Integración Continua (CI/CD Pipeline)

Render monitorea en tiempo real la rama principal (`main`) del repositorio de GitHub. El flujo automatizado se ejecuta en 4 fases:

```
[ Git Push origin main ]
    |
    ▼
[ Trigger Webhook en Render ]
    |
    ▼
[ Build Phase: npm install & ng build / npm run build ]
    |
    ▼
[ Release Phase: Inyección de Variables de Entorno (.env) ]
    |
    ▼
[ Start Phase: process.env.PORT & node index.js ] → ( Live Web Application )
```

1. **Trigger de Commit:** Cualquier actualización subida al repositorio mediante `git push origin main` dispara automáticamente el proceso de construcción en Render.

2. Fase de Compilación (*Build Command*):

```
`bash
```

```
npm install && npm run build
```

```
`
```

Instala dependencias y genera el paquete optimizado listo para producción.

3. Fase de Inicio (*Start Command*):

```
`bash
```

```
node index.js
```

```
`
```

Ejecuta el servidor Express backend.

3.3. Configuración de Variables de Entorno en el Dashboard de Render

Para garantizar la seguridad de la aplicación en producción, **ningún secreto o contraseña está escrito en duro en el código fuente**. En el panel de Render (*Environment Variables*) se configuran los valores reales:

Variable de Entorno	Descripción / Valor en Producción
---------------------	-----------------------------------

:---	:---
------	------

PORT	Asignado dinámicamente por Render (ej: 10000)
------	--

DB_HOST	Host del servidor de PostgreSQL en la nube
---------	--

DB_USER	Usuario de base de datos de producción
---------	--

DB_PASSWORD	Contraseña cifrada de la base de datos
-------------	--

DB_NAME	unamconnect_db
---------	----------------

DB_PORT	5432
---------	------

JWT_SECRET	Firma criptográfica de alta seguridad para Tokens JWT
------------	---

FIREBASE_PROJECT_ID	ID del proyecto en Google Firebase Console
---------------------	--

FIREBASE_CLIENT_EMAIL	Correo de la cuenta de servicio de Firebase
-----------------------	---

FIREBASE_PRIVATE_KEY	Clave privada RSA de Firebase Admin SDK
----------------------	---

3.4. Consideraciones Técnicas Críticas para Producción

1. Manejo de Puerto Dinámico (`process.env.PORT`):

En `backend/index.js`, el servidor no escucha en un puerto estático, sino que respeta el asignado por Render:

```
`javascript

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {

  console.log( Servidor UNAMConnect escuchando en el puerto ${PORT} );

});`
```

2. Políticas CORS (Cross-Origin Resource Sharing):

Se configuró Express para autorizar el intercambio de recursos entre el cliente Angular y la API REST en producción, evitando bloqueos del navegador.

3. Despliegue sin Tiempo de Inactividad (Zero Downtime Deployment):

Render mantiene la versión previa en línea mientras construye la nueva versión. Una vez completado el build de forma exitosa, conmuta el tráfico al nuevo contenedor de manera transparente.

4. Guion de Exposición para la Sustentación 🎤

Utiliza este guion estructurado para explicar la infraestructura Cloud ante el docente o jurado:

Paso 1: Explicación de los Servicios Cloud (2 mins)

"Buenas tardes profesor/jurado. En esta sección explicaremos la infraestructura Cloud que da soporte a UNAMConnect. Para garantizar un sistema seguro, rápido y escalable, integramos dos servicios de clase mundial: Google Firebase para la gestión de archivos y autenticación delegada, y la plataforma PaaS Render para el despliegue automático en la nube."

Paso 2: Integración con Firebase (2 mins)

"Utilizamos Firebase de dos formas: en el cliente frontend para permitir que los usuarios soliciten el restablecimiento de contraseñas de manera segura, y Firebase Storage para el almacenamiento de archivos pesados. Como las boletas de notas que adjuntan los postulantes a tutor son documentos críticos, se suben directamente a la nube de Firebase generando URLs públicas seguras con redundancia geográfica. Nuestro backend en Node.js se conecta usando Firebase Admin SDK reconociendo dinámicamente si estamos en desarrollo local o en producción mediante variables de entorno."

Paso 3: Demostración del Despliegue en Render y CI/CD (3 mins)

"Nuestra aplicación se encuentra totalmente desplegada y accesible en la web en <https://unamconnect.onrender.com> . El proceso de despliegue está automatizado mediante un pipeline de Integración Continua (CI/CD). Cada vez que realizamos un `git push origin main` a nuestro repositorio de GitHub, Render detecta el cambio, instala dependencias, ejecuta la compilación de Angular y levanta el servidor Express sin interrumpir el servicio a los usuarios."

Paso 4: Respuestas a Preguntas Probables del Jurado

Pregunta: ¿Por qué no guardaron las boletas de notas de los estudiantes en el mismo servidor Node.js?

> "Respuesta: Profesor, los servidores de aplicaciones en la nube como Render utilizan un sistema de archivos efímero (Ephemeral Storage). Si guardáramos las boletas en el disco local del servidor, al reiniciarse la aplicación o desplegar una nueva versión, los archivos se perderían. Firebase Storage nos garantiza almacenamiento persistente e indestructible en la nube."

Pregunta: ¿Cómo protegen las llaves de acceso a Firebase en el código del repositorio en GitHub?

> "Respuesta: Aplicamos las mejores prácticas de seguridad en desarrollo de software. Las llaves privadas de Firebase y la contraseña de la base de datos están excluidas del control de versiones a través de `.gitignore` y se inyectan exclusivamente como variables de entorno seguras desde el panel de administración de Render."